# Cryptography & Network Security

ETTCS801 Integrated Situation — Technical Report

UMWUNGERI Jean Robert Clement
4202670015
ULK Polytechnic Institute

September 28, 2026

## Contents

**1 Introduction** **2**

**2 Risk Assessment** **2**
2.1 Assets, Vulnerabilities and Consequences . . . . . 2
2.2 Risk Ranking . . . . . 2
2.3 Recommended Controls . . . . . 2

**3 Encryption, Decryption and Integrity** **3**
3.1 Design . . . . . 3
3.2 Test Evidence . . . . . 3

**4 Network Traffic Filtering** **4**
4.1 Laboratory Setup . . . . . 4
4.2 Firewall Rules . . . . . 4
4.3 Test Procedure and Results . . . . . 4

**5 Conclusion** **5**

# 1 Introduction

This report documents the work completed for the ETTCS801 Integrated Situation. The institute stores student records on a central server and transfers files between two campuses. A security review found weak staff passwords, outdated software, unencrypted file transfers and guest network access to the records server, and the IT team observed repeated connection attempts from an unfamiliar external address. The work consists of a risk assessment, a Python encryption and integrity toolkit, and firewall rules that protect the records server.

# 2 Risk Assessment

## 2.1 Assets, Vulnerabilities and Consequences

| Asset | Vulnerability | Possible consequence |
|---|---|---|
| Central student records server | Guest network can reach the server | An unauthorised or compromised guest device could read, copy or alter student records. The repeated external connection attempts show the server is already being probed. |
| Staff accounts and authentication | Weak staff passwords | An attacker who guesses a password gains trusted access to the records server and bypasses perimeter controls. |
| Inter-campus file transfer channel | Unencrypted transfers and outdated software | Data in transit can be intercepted or altered, and known unpatched flaws may allow deeper access once an attacker has a foothold. |

## 2.2 Risk Ranking

| Rank | Risk | Likelihood | Impact | Reason |
|---|---|---|---|---|
| 1 | Guest access to records server | High | High | The threat is already observed: repeated connection attempts give an attacker a direct path to the asset. |
| 2 | Weak staff passwords | High | High | Weak passwords are easy to guess or reuse, and a stolen staff account gives trusted access. |
| 3 | Unencrypted transfers and outdated software | Medium | Medium–High | Interception needs a position on the transfer path, a higher bar, but outdated software raises the impact if a foothold is gained. |

## 2.3 Recommended Controls

- **Guest access:** network segmentation with firewall rules that deny the guest subnet and permit only the authorised staff subnet (implemented in Section 4).
- **Weak passwords:** a strong password policy (length, complexity, no reuse) combined with multi-factor authentication.

- **Unencrypted transfers and outdated software:** encrypted transfer (SFTP, SCP or TLS) and a regular patch-management schedule.

# 3 Encryption, Decryption and Integrity

## 3.1 Design

The toolkit `encryption_tool.py` is a command-line program with five commands: `genkey`, `encrypt`, `decrypt`, `hash` and `verify`. It uses Fernet from Python's `cryptography` library, which combines AES-128 in CBC mode with an HMAC, so encrypted files are both confidential and tamper-evident.

- **Encryption:** `encrypt_file` reads the plaintext, encrypts it with the key and writes the ciphertext to a new file.
- **Decryption and verification:** `decrypt_file` recovers the plaintext and, with `--compare-to`, compares it byte for byte with the original.
- **Integrity:** `hash_file` computes a SHA-256 digest and can save it; `verify_integrity` recomputes the digest later and reports whether the file has changed.
- **Key handling:** the key is stored outside the repository (default `../secret.key`) and `.gitignore` excludes key files.
- **Error handling:** a missing file, empty file, unreadable file, invalid key, wrong key or tampered ciphertext produces a clear `[ERROR]` or `[FAIL]` message and a non-zero exit code, never an unhandled exception.

## 3.2 Test Evidence

The following output was produced by running the toolkit on the sample record. The first listing shows encryption, decryption with byte-for-byte verification, hashing and an unchanged-file check; the second shows tamper detection and the error-handling cases.

```
$ python3 encryption_tool.py genkey --key ../secret.key
[OK] New key written to: ../secret.key
     Keep this file private and outside the Git repository.

$ python3 encryption_tool.py encrypt sample_data/sample_student_record.txt sample_data/
     sample_student_record.enc --key ../secret.key
[OK] Encrypted 'sample_data/sample_student_record.txt' -> 'sample_data/sample_student_record.enc'
     SHA-256 of original file: 4649b4dfa7841a5d4023b17923dec73ddacada717c484cda2241b9aab8caae4d

$ python3 encryption_tool.py decrypt sample_data/sample_student_record.enc sample_data/
     sample_student_record.dec --key ../secret.key --compare-to sample_data/sample_student_record.txt
[OK] Decrypted 'sample_data/sample_student_record.enc' -> 'sample_data/sample_student_record.dec'
[OK] Verification PASSED: decrypted content matches the original file.

$ python3 encryption_tool.py hash sample_data/sample_student_record.txt --save sample_data/
     sample_student_record.sha256
[OK] SHA-256(sample_data/sample_student_record.txt) = 4649
     b4dfa7841a5d4023b17923dec73ddacada717c484cda2241b9aab8caae4d
     Hash saved to: sample_data/sample_student_record.sha256

$ python3 encryption_tool.py verify sample_data/sample_student_record.txt sample_data/sample_student_record
     .sha256
     Stored hash : 4649b4dfa7841a5d4023b17923dec73ddacada717c484cda2241b9aab8caae4d
     Current hash: 4649b4dfa7841a5d4023b17923dec73ddacada717c484cda2241b9aab8caae4d
[OK] Integrity check PASSED: file has not changed.
```

```
$ python3 encryption_tool.py verify sample_data/tampered_copy.txt sample_data/sample_student_record.sha256
     Stored hash : 4649b4dfa7841a5d4023b17923dec73ddacada717c484cda2241b9aab8caae4d
     Current hash: c342246ef28b9a3233561f0a854b59e5e0af371d341b2db7bd85ce231d402343
```

```
[FAIL] Integrity check FAILED: file has been modified since the hash was recorded.

$ python3 encryption_tool.py hash sample_data/does_not_exist.txt
[ERROR] File not found: sample_data/does_not_exist.txt

$ python3 encryption_tool.py decrypt sample_data/sample_student_record.enc sample_data/bad.dec --key ../
    wrong.key
[ERROR] Decryption failed: wrong key, or the file is corrupted/tampered with.

$ python3 encryption_tool.py encrypt sample_data/sample_student_record.txt sample_data/x.enc --key ../
    no_such.key
[ERROR] Key file not found: ../no_such.key
       Run 'python encryption_tool.py genkey' first.
```

# 4 Network Traffic Filtering

## 4.1 Laboratory Setup

Only one machine was available, so three separate networks were simulated on it using Linux network namespaces connected by virtual Ethernet pairs (`netns_setup.sh`). The host acts as the records server and runs the firewall.

| Network | Subnet | Role |
|---|---|---|
| `staff_ns` | 192.168.10.0/24 | Authorised staff |
| `guest_ns` | 192.168.50.0/24 | Guest network |
| `other_ns` | 10.99.99.0/24 | Other external source |

The protected service is TCP port 8000, provided by a Python HTTP server on the host. If the assessor specifies different values, they are changed at the top of `firewall_rules.sh`.

## 4.2 Firewall Rules

```
sudo ufw deny from 192.168.50.0/24 to any port 8000 proto tcp # 3a guest blocked
sudo ufw allow from 192.168.10.0/24 to any port 8000 proto tcp # 3b staff permitted
sudo ufw deny 8000/tcp # 3c all others blocked
```

Rules are matched in order, so the specific guest and staff rules take effect before the general deny. The resulting rule set is confirmed with `sudo ufw status numbered`:

```
$ sudo ufw status numbered
Status: active

     To Action From
     -- ------ ----
[ 1] 8000/tcp DENY IN 192.168.50.0/24
[ 2] 8000/tcp ALLOW IN 192.168.10.0/24
[ 3] 8000/tcp DENY IN Anywhere
[ 4] 8000/tcp (v6) DENY IN Anywhere (v6)
```

## 4.3 Test Procedure and Results

One permitted and two blocked connections were tested, each from inside the relevant namespace.

| Test | Command | Expected | Actual |
|---|---|---|---|
| Staff (permitted) | `ip netns exec staff_ns curl http://192.168.10.1:8000/` | HTTP 200 | HTTP 200, exit 0 |

| Test | Command | Expected | Actual |
|---|---|---|---|
| Guest (blocked) | `ip netns exec guest_ns curl http://192.168.50.1:8000/` | Timeout, no response | HTTP 000, exit 28 (timeout) |
| Other (blocked) | `ip netns exec other_ns curl http://10.99.99.1:8000/` | Timeout, no response | HTTP 000, exit 28 (timeout) |

```
$ sudo ip netns exec staff_ns curl --max-time 3 -s -o /dev/null -w "HTTP %{http_code}\n" http
    ://192.168.10.1:8000/
HTTP 200
curl exit code: 0

$ sudo ip netns exec guest_ns curl --max-time 3 -s -o /dev/null -w "HTTP %{http_code}\n" http
    ://192.168.50.1:8000/
HTTP 000
curl exit code: 28

$ sudo ip netns exec other_ns curl --max-time 3 -s -o /dev/null -w "HTTP %{http_code}\n" http
    ://10.99.99.1:8000/
HTTP 000
curl exit code: 28

Server log (only the staff request arrived):
192.168.10.2 - - [28/Sep/2026 14:41:41] "GET / HTTP/1.1" 200 -
```

# 5 Conclusion

The assessment identified guest network exposure, weak staff passwords, and unencrypted transfers with outdated software as the main risks to the student records system, ranking guest exposure highest because probing was already observed. The Python toolkit provides confidential storage of records through encryption, confirms that decryption restores the original data, and detects later modification through SHA-256 hashing, while handling invalid input without failing. The firewall rules restrict the records service to the authorised staff network and block guest and all other sources. A limitation is that the networks were simulated on a single machine rather than a physical segmented network, although the firewall matching logic is the same. Password policy, multi-factor authentication, encrypted transfers and patching remain to be implemented as organisational controls.

GitHub repository: `https://github.com/hnmarc-hub/cryptography-network-security-exam`